

Highlights

From a Production OTP Flood to a Layered Defence: Measured Limits of Risk-Scored SMS Verification Against Residential Flooders and Colluding Carriers

Naveed Ahmed

- A production OTP flood as the operator recalls it, the first fix, and the attack that returned
- Signature attacks are contained with 99 % of simulated real users delivered
- A human-like residential flooder is held only by rationing; the trade-off is measured
- Pumping on few destination blocks leaks about five messages per block, by model and by test
- The block tests' false positives, poisoning and outage behaviour set their memory setting

From a Production OTP Flood to a Layered Defence: Measured Limits of Risk-Scored SMS Verification Against Residential Flooders and Colluding Carriers

Naveed Ahmed^{a,*}

^a*TODO: affiliation, TODO, TODO, Saudi Arabia*

Abstract

A registration endpoint that sends a one-time password (OTP) by SMS to any number it is given will be used to send SMS at the operator's expense, either to waste money (flooding) or to earn a share of the termination fee (SMS pumping). We describe such an incident as the operator recalls it, the controls that stopped it, and how the attack returned through residential proxies, a forged platform header and per-key thresholds. We then present a twelve-step pipeline that derives trust from attestation, signed sessions and verification outcomes rather than from client claims, and evaluate it in a seeded simulation with legitimate traffic in the background, against six simpler designs, nine attacker profiles, nine adaptive attackers and four colluding-carrier pumpers. No production traffic was available; every absolute figure is conditional on the stated calibration. Attacks that leave a signature in the request are contained: datacenter rotation and premium-range pumping leak nothing, a sequential walk leaks about 70 of 600 messages. A residential attacker with farmed CAPTCHA scores and valid numbers is not separated by any signal we tested; an adaptive volume cap holds it to 243 messages by refusing 51 % of first-time users, and only when the cap is recomputed every minute. Against pumping concentrated on a few destination blocks, a sequential test keyed on the 10 000-number block leaks about five messages per block plus the in-flight window, to within 19 messages of a closed-form estimate at every configuration measured, until the carrier verifies its own codes or fakes delivery receipts. The tests' cost to real users

*Corresponding author.

Email address: `optimistbs@gmail.com` (Naveed Ahmed)

is measured over 24-hour runs, a carrier outage and a deliberate poisoning attack, and it fixed the detector’s memory setting. Code, evaluation and recorded results are public.

Keywords: SMS pumping, artificially inflated traffic, one-time password, rate limiting, risk scoring, residential proxies, sequential tests, fraud prevention

1. Introduction

Most online services verify a phone number the same way: the user types the number, the service sends it a code by SMS, the user types the code back. The sending step costs money, between a tenth of a cent and twenty cents per message depending on the destination, and the endpoint that triggers it is reachable before any account exists. A flooder can make the service send thousands of messages to numbers it does not own. A pumper does the same to numbers on a range whose operator shares the termination fee with it, which turns the service’s SMS budget into the attacker’s income; MITRE lists the technique as resource hijacking [1]. In 2023 Twitter’s owner stated that the company had been losing about 60 million dollars a year to operators pumping two-factor SMS [2], and Huh et al. found that a three-month log of one large authentication service held as many attack requests as genuine ones in the nineteen countries they studied [3].

This paper starts from such an incident on a registration flow one of us operated. The first response was conventional: a hidden CAPTCHA, one message per number per minute, five requests per address per minute, a country allow-list, a limit on message length and a block on proxy connections. It stopped the attack it was built for. The attack that came back rotated through residential addresses, sent the header that marked a request as coming from the iOS app, and kept every number and address under its threshold. Reviewing the first design against that attacker produced eight named gaps, and the second design, described here, is built around them.

The second design is a pipeline of twelve steps and an asynchronous feedback loop. Platform exemptions are granted only after Apple App Attest or Google Play Integrity has verified the app. Every client carries a signed session token bound to a browser fingerprint, so that an identity persists across an address change (though, as Section 4 is careful to say, the signature protects the token, not the uniqueness of the fingerprint). Numbers are checked

30 for country, prefix cost class, generated patterns and liveness before a mes-
31 sage is paid for. Each network, device, country and number range carries a
32 reputation derived from whether the codes it asked for were entered. A risk
33 score built on these signals chooses among five responses, and a hard hourly
34 budget bounds total spend.

35 We implemented it and evaluated it in simulation, because no production
36 logs were available to us. The evaluation is organised around five questions.

37 RQ1 Which attacks does the design contain, compared with six simpler de-
38 signs, and at what cost to simulated real users?

39 RQ2 Which layer is responsible, measured as paired differences on the same
40 workload?

41 RQ3 When does reputation built on verification outcomes separate an at-
42 tacker from real users in this design, and when does it not?

43 RQ4 When it does not, what does rationing cost, and what does it depend
44 on?

45 RQ5 What do tests that act on a shared key, the destination number block,
46 cost real users when the population differs from the calibration, a carrier
47 fails, or an attacker aims them at real users?

48 The answers narrow the design’s claims, and the paper is written around
49 that: signature attacks are contained; a residential attacker that looks like a
50 real user is held only by a volume cap whose cost and cadence dependence we
51 measure; pumping concentrated on a few destination blocks is contained by
52 a sequential test on the block, at a cost a closed-form estimate reproduces,
53 until the carrier verifies its own codes, builds trust first or fakes its receipts,
54 and the same test can be turned on real users.

55 The contributions are:

- 56 1. An operator’s account of an OTP flood, the controls that stopped it,
57 and the adapted attack that defeated them, as eight named gaps (Sec-
58 tion 3).
- 59 2. A layered design with graded responses and an open implementation,
60 evaluated against six simpler designs including a corrected version of
61 the first one, a budget-only controller and a flat per-block limit (Sec-
62 tions 4, 6 and 7.1).

- 63 3. An empirical separability result for outcome-based reputation in this
64 design: it separates an attacker only on keys the attacker dominates,
65 shown by a dilution curve, a parameter sweep and a cadence study
66 (Section 7.3).
- 67 4. A closed-form estimate of pumping leakage against destination-block
68 tests, checked against observed block counts, with its evasion condi-
69 tions and the memory setting that trades evasion against false positives
70 (Sections 5 and 7.5).
- 71 5. A collateral-damage study of destination-keyed reputation: miscali-
72 brated populations, a carrier outage and a deliberate poisoning attack
73 (Section 7.6).

74 Section 2 places the work; Sections 3 to 6 give the incident, the design,
75 the model and the method; Section 7 answers the questions; Section 8 states
76 lessons and limitations.

77 2. Background and related work

78 2.1. *Where the money goes*

79 Telephony fraud depends on the gap between what a call or message
80 costs to deliver and what someone is paid for terminating it. Sahin et al.
81 organised the field into root causes, weaknesses, techniques and schemes, and
82 placed international revenue share fraud (IRSF) among the revenue-share
83 schemes: a party advertises a range of numbers as premium destinations,
84 traffic is generated to them, and the termination fee is split along the route [4].
85 They note that the causes, legacy infrastructure, routing where each operator
86 knows only the next hop, and operators' reluctance to share fraud findings,
87 are unlikely to be fixed soon. SMS is a minor topic in that survey.

88 Sahin and Francillon later collected 3.14 million test numbers advertised
89 by twelve IRSF providers over four years and showed that the number range
90 is the fraudster's managed asset: about 80 % of ranges were shared across
91 providers, 57 % were mobile ranges, and one block of a thousand Latvian
92 numbers was carved into sub-ranges by six providers in one year [5]. They
93 also showed why acting on a range is dangerous: coarse blocking of IRSF
94 destinations has made whole countries unreachable, and their own range-
95 based detector had a 5.8 % false-positive rate against 0.26 % for a classifier on
96 call features. Both points carry over to SMS pumping and shape Sections 4.4
97 and 7.6.

2.2. Detecting artificially inflated SMS traffic

The closest work is by Huh et al. [3]. They had 9.4 million SMS authentication logs from a service with tens of millions of users, nineteen countries, three months of 2023, labelled by a peak-traffic heuristic into 4.95 million attack and 4.50 million genuine requests. Gradient-boosted trees on per-request, per-entity and country-wide features reached a recall of 89.6 % at a false-positive rate of 0.19 % on web traffic and 91.1 % at 0.11 % on native traffic; the system is deployed and blocks above a threshold. Three findings shaped our design. Per-entity limits do not work: about 73 % of web attacks sent four or fewer requests per identity in 24 hours. The strongest signals were the destination’s: the number of distinct numbers sharing a prefix in a day was the dominant native feature, and attacks showed about 92 % zero conversion against 21 % for genuine traffic. And an attacker would need about 48 times as many prefixes to evade the web model.

We did not reproduce their detector, and we say so: it needs labelled traffic we do not have, and a supervised model on synthetic features would only re-learn our own generator. What this paper adds beside it is narrower than “a pipeline”. Their system answers block or allow per request from learned features; ours asks, for one unlearned feature family, verification outcome per key, where it can separate at all (Section 7.3) and what a sequential test on the destination costs real users (Section 7.6). Their non-peak attacks, on which they report low recall, are the low-and-slow profiles of Section 7.7. We regard a trained model of their kind as a candidate input to the risk score, not as a competitor to the limits around it.

2.3. Defences in practice and their known limits

Commercial SMS verification services ship two kinds of filter. Twilio’s Verify Fraud Guard blocks verifications to regions known for pumping and destinations new relative to an account’s history, at three protection levels with stated false-positive rates under 1 % and under 2 % [6]. Twilio’s SMS Pumping Risk Score is closer to what we build: a per-number lookup made before each OTP request, returning a 0–100 score from “signals from Verify Fraud Guard and risky carriers, unusual SMS traffic patterns, and low conversion rates”, with graded guidance, send below 60, add friction between 60 and 90, do not send above 90 [7]. A risk score with graded action on the destination is therefore established commercial practice; what the documentation does not give is the model, its memory, or its false-positive behaviour, which is what Sections 4.4, 7.5 and 7.6 measure for one explicit design.

135 The client-side controls that services deploy first have measured limits.
 136 Motoyama et al. showed in 2010 that CAPTCHA solving is a labour market,
 137 with retail prices of a dollar per thousand, 86 to 89 % accuracy and median
 138 response times of ten to twenty seconds, and argued that a CAPTCHA is
 139 friction on the attacker’s economics rather than a barrier [8]. Dionysiou and
 140 Athanasopoulos classified the automated solvers and concluded that machine
 141 learning alone was not sufficient to break text CAPTCHAs, while the human
 142 route remained open [9]. Client puzzles price a request in computation in-
 143 stead [10]; we did not evaluate them, and Section 8 lists that as a gap. On
 144 the address side, Mi et al. infiltrated five residential proxy services and found
 145 6.18 million addresses, 95 % residential, 90 % relaying for under fifteen min-
 146 utes before rotating, 2.2 % on any blacklist [11]. Richter et al. measured
 147 that more than 90 % of cellular networks sit behind carrier-grade NAT, with
 148 up to 128 subscribers behind one address, and concluded that address-based
 149 reputation is “all but infeasible” there [12].

150 *2.4. Identity, sequential tests and the security of codes*

151 Douceur showed that without a logically centralised authority an entity
 152 can always present multiple identities except under unrealistic assumptions
 153 of resource parity [13]. That is the position of a web client here: nothing
 154 scarce backs a browser fingerprint, so a signed session establishes continuity
 155 of one token, not scarcity of identities, and the design’s per-identity controls
 156 can only tax, not prevent, identity reset. App attestation is the one place
 157 the design has a certifying authority, and only for app clients.

158 Our destination tests are sequential probability-ratio tests after Wald [14],
 159 run as Page’s cumulative sums with a bounded floor [15]. Howard et al.
 160 give time-uniform confidence sequences that remain valid under continuous
 161 monitoring and optional stopping and generalise the SPRT to nonparametric
 162 settings [16]; we use the classical parametric form and report its error rates
 163 empirically rather than claiming time-uniform guarantees, which Section 8
 164 lists as the right next step.

165 Risk-based authentication scores a login from implicit features; Wiefling
 166 et al. found that every service applying it used the IP address and that single-
 167 feature deployments re-authenticated users often [17]. Step 7 is a risk engine
 168 of that kind applied before a message is sent. The security of the code once
 169 sent is a separate problem [18, 19]: those attacks target the person who asked
 170 for a code, ours the service that sends it.

171 3. The incident and the adapted attacker

172 3.1. What happened, as the operator recalls it

173 We have no logs from the period and give no request counts or bill figures;
174 this section is an operator’s account and motivates the design, nothing more.
175 The registration flow sent an SMS to any unregistered number it was given.
176 During its first weeks the endpoint was called repeatedly with generated
177 numbers in the allowed country and every call produced a paid message.
178 There was no limit per number, per address or in total, no check on the
179 body, no country restriction and nothing that told a browser from a script.

180 The first controls, v1, are in Table 1: a hidden reCAPTCHA v3 with
181 a score threshold on web requests, one message per number per minute,
182 five requests per address per minute, a country allow-list with one coun-
183 try exempted for app users, a body-length limit, a block on connections a
184 proxy-detection service flagged, and per-platform, per-country caps tuned
185 to historical peaks. The abuse that returned did not trip them. Requests
186 arrived from thousands of residential addresses, each used a few times, car-
187 ried the iOS platform header and so skipped the CAPTCHA and the host
188 check, and kept each number and address under its limit. Table 1 lists the
189 eight gaps that review exposed. Two of them, trusting a client header and
190 a check-then-increment limit, are design defects by any standard; the others
191 are assumptions that held for the first attacker and not the second.

Table 1: The v1 controls and the gaps the adapted attacker exposed.

Gap	v1 assumed	How it was bypassed
A	The HTTP_PLATFORM header says which platform sent the request; iOS skips the host check, apps skip reCAPTCHA	Sending the header. Every platform exemption was reachable by anyone
B	Five requests per address per minute bounds one attacker	Residential proxies give a fresh address per request; nothing keyed the attacker across addresses
C	A number passed to the provider is worth sending to	Random and sequential numbers cost the same as real ones; nothing checked liveness
D	An allowed country is a safe destination	Premium-rate and revenue-share ranges inside the country cost many times a standard message
E	Per-key caps bound total spend	A wide attack under every cap burned budget without bound
F	Check-then-increment is a rate limit	Two instances racing both passed
G	A bulk flag and a test-server flag are internal	Neither required stronger authentication than a normal request
H	Each check is pass or fail and the response says which failed	Thresholds were probed one at a time; registered numbers could be enumerated

192 3.2. Threat model

193 The attacker wants either to make the service send as many messages
194 as possible or to make it send messages to numbers on which the attacker
195 is paid. It can buy residential proxy access at about 3 USD per gigabyte,
196 a fresh browser fingerprint at no cost, and reCAPTCHA v3 tokens with
197 human-like scores at about 0.3 cents each; it can generate numbers at will
198 and knows the service’s thresholds. A pumper additionally controls, through
199 a colluding operator or reseller, a set of destination ranges, may read the
200 messages delivered there and enter the codes, may report deliveries as failed,
201 and may spend time building verified history before attacking. Interactive
202 challenges can be bought solved. We assume the attacker cannot forge an app
203 attestation, cannot change which ranges its partner terminates, and cannot
204 make a real person on an unrelated number enter a code. Whether a message
205 the carrier reports as undelivered is still billed and revenue-bearing varies by
206 contract; we assume it is, which is the case that favours the attacker. Table 4
207 lists the profiles built from these capabilities.

208 4. Design

209 Two principles drive the design. Cheap local checks run first, so that
210 obvious abuse is refused before a vendor call and long before a message is
211 paid for. Trust is derived from things the client cannot set: attestation, a
212 server-signed session, and what happened to the codes it asked for before.

213 4.1. *The pipeline*

214 Every OTP request passes through twelve steps (Table 2); the first failing
215 step stops it. Steps 0 to 6 are hard rejects on cheap evidence; Step 7 turns
216 the evidence into a score and a tier; Steps 8 to 10 are the capacity controls;
217 Step 11 picks a channel, writes the audit record and answers.

Table 2: The twelve steps. The last column names the gap each step was built against; whether it closes it is what Section 7 measures. FB is the feedback loop.

Step	Check	Built against
0	Proxy block; platform derived from attestation, never from the header	Gap A
1	Signed session token bound to a fingerprint; nonce replay; 3 requests per session per 10 min	Gap B, replay
2	Atomic limits per address, per /24 subnet and per ASN, lower for hosting ASNs	Gaps B, F
3	reCAPTCHA v3 with a score threshold, web only	bots
4	Origin check, web only	domain allow-list
5	Number intelligence: E.164 parsing, country, prefix cost class, sequential and narrow-range detection, liveness lookup; bulk bypass needs a service credential	Gaps C, D, G
6	Body length; excluded numbers; test-server bypass needs a service credential	body limits, Gap G
7	Risk score and tier: allow, delay, challenge, downgrade, block	Gap H
8	Per-number backoff, 60 s doubling to 1 h, five per day; window and daily slot claimed atomically and released if a later step refuses	per-number flooding
9	Per-source, per-platform, per-country caps with an adaptive multiplier; a downgraded request consumes none of it	aggregate caps
10	Hourly count and spend budgets reserved atomically before a send; a send over either is not made	Gap E
11	Channel selection; audit log; one response body for every outcome after Step 0 except the challenge; a 400 ms minimum response time	Gap H
FB	Verification feedback: verified, failed, undelivered per key; destination-block tests; denylist of fine keys	conversion as a signal

218 Three details matter for what follows. Step 0 never reads the platform
219 from the header: an iOS request is one with a valid App Attest assertion, an
220 Android request one with a valid Play Integrity verdict, and anything else
221 is web traffic with no exemption, so a forged header is ignored rather than
222 refused. Step 2’s limits are each one Lua script in Redis; the per-number
223 claim in Step 8 and the budget reservation in Step 10 are likewise atomic,
224 and a claim is released if a later step refuses. Step 11 returns HTTP 200 with
225 one body for every outcome after Step 0 except the interactive challenge, and

226 pads the handler to a 400 ms minimum; Step 0 returns 403 to proxies and
 227 failed attestations. What that contract hides from an attacker is measured
 228 in Section 7.7, not assumed.

229 4.2. The risk score and its tiers

230 Step 7 computes

$$s = \sum_i w_i x_i + w_c \frac{\max(0, \theta - r_{\min})}{\theta} + w_s f + w_m i - w_v v_n - w_f v_f, \quad (1)$$

231 where the x_i are the per-request signals of Table 3, $r_{\min}$ is the lowest con-
 232 version ratio over the keys the request touches (address, subnet, ASN, fin-
 233 gerprint, country, prefix and destination block), $\theta = 0.3$, $w_c = 25$, f flags
 234 a sustained flood on any key, i flags a key whose codes are mostly entered
 235 within seconds of delivery, and v_n , v_f indicate a number and a fingerprint
 236 with verified history. The score is clamped to $[0, 100]$ and mapped to a tier at
 237 20, 40, 60 and 80: *allow* sends at once; *delay* sends after a few seconds that
 238 double per request in the session; *challenge* returns an interactive challenge
 239 to a browser and re-enters Step 7 with a -20 credit when it is solved (apps
 240 are downgraded instead); *downgrade* may use push, WhatsApp or silent net-
 241 work authentication but never SMS; *block* refuses. The signals are correlated
 242 (a datacenter address and a fresh fingerprint often arrive together) and the
 243 weights were set by hand and then swept (Section 7.3); the score is a linear
 244 tally, not a calibrated probability.

Table 3: Signals and weights in the risk score. Positive values raise risk.

Signal	Points
reCAPTCHA score c (web)	$(1 - c) \times 25$
Address in a hosting or datacenter ASN	+15
Address abuse-list score $a \in [0, 1]$	$a \times 15$
Fingerprint first seen under 5 min ago / under 1 h ago	+20 / +10
Each previous request in this session	+5
Lowest conversion ratio over the request's keys, when below 0.3	up to +25
Sustained flood on a key (≥ 100 resolved sends, ratio < 0.1)	+15
Instant verification on a key (≥ 20 verified, $> 80\%$ within 5 s of delivery)	+15
Address country differs from number country	+10
Sequential number; narrow-range burst	+15 each
Elevated-cost prefix; unknown prefix; VoIP number	+10 each
App without attestation inside the grace window	+20, never better than delay
Interactive challenge solved	-20
Number verified before; fingerprint verified before	-20; -15

Two consequences of the weights matter later. The conversion penalty attaches to keys real users share with whoever is attacking through them, so on its own it moves a client only to *delay*, which still sends. A datacenter address with a fresh fingerprint and a foreign country scores about 47 even with a good CAPTCHA, which is *challenge*, not *block*: datacenter rotation is stopped because that attacker does not solve challenges, and Section 7.7 shows what happens when it does.

4.3. The feedback loop

Each send is attached to the reputation keys of its request. A send resolves as *verified* when the code in the message is entered; as *failed* when five wrong attempts are made or two minutes pass after the provider's delivery receipt with no attempt; as *undelivered* when no receipt arrives within sixty seconds of the send or the receipt reports failure. Only verified and failed feed the conversion ratio and the block tests. The two-minute window replaced the ten-minute code validity after we found that 99% of simulated users verify within about 100 seconds of delivery; a late verification is reclassified. Verification speed is clocked from the receipt. A provider without delivery reports is configured to clock from the send, as our first version did, and

263 Section 7.6 shows the difference. Fine keys whose ratio falls below 0.1 over
 264 fifty resolved sends go on a 24-hour denylist: addresses, subnets, fingerprints
 265 and hosting ASNs, never a residential ASN.

266 4.4. *Destination-block tests*

267 Every key above can be rotated by the attacker. The destination is the one
 268 a pumper cannot: it is paid only on the numbers its partner terminates. We
 269 therefore also key reputation on the 8-digit block of the destination, 10 000
 270 numbers, and judge blocks by sequential tests rather than by the shared-key
 271 thresholds, since a block is touched by legitimate traffic rarely (the 24-hour
 272 study uses 4, 29 and 144 sends per block per day) and a block with several
 273 sends in a few minutes is usually one party's.

274 Two statistics accumulate per block, each the log-likelihood ratio of at-
 275 tacker to legitimate behaviour. The conversion statistic assumes a real user
 276 verifies with probability $p_\ell = 0.8$ and a flooder's number with $p_a = 0.1$;
 277 a failure adds $\ln \frac{1-p_a}{1-p_\ell} = 1.50$ and a verification adds $\ln \frac{p_a}{p_\ell} = -2.08$. The
 278 speed statistic assumes a machine enters a code within five seconds of deliv-
 279 ery with probability $q_a = 0.9$ and a person with $q_\ell = 0.2$, the latter because
 280 Android's SMS Retriever and the WebOTP API fill the code in without
 281 a keystroke [20, 21]; an instant verification adds $\ln \frac{q_a}{q_\ell} = 1.50$ and a slow
 282 one $\ln \frac{1-q_a}{1-q_\ell} = -2.08$. A verdict is reached when either statistic exceeds
 283 $\ln 1000 = 6.91$. Both p_ℓ and q_ℓ are parameters a deployment must set from
 284 its own traffic; Section 7.6 measures what happens when they are wrong.

285 The statistics are floored, which is the detail the evaluation turned on.
 286 A plain SPRT lets a block bank unbounded credit: after a hundred verified
 287 sends the conversion statistic sits near -200 and 143 failures are needed to
 288 reach a verdict, so a carrier that verifies for a while and then floods is never
 289 caught. Page's cumulative sum floors the statistic at zero and banks nothing,
 290 and catches that carrier, but it also turns a legitimate block converting at
 291 65% into a hundred false verdicts a day (Section 7.6). The deployed default
 292 floors each statistic at minus one threshold, -6.91 : a block can bank at most
 293 one verdict's worth of goodwill. Section 7.5 reports all three settings side by
 294 side.

295 What a verdict does is graded. The first verdict makes the block's first-
 296 time clients solve an interactive challenge for an hour; a second inside that
 297 hour moves them to non-SMS channels. Clients with verified history are not
 298 affected. A 24-hour denylist is kept as an option. The tests are suspended for
 299 fifteen minutes on a prefix whose delivery receipts collapse across at least ten

300 blocks, or whose returning clients stop verifying across at least ten blocks;
 301 only clients with verified history count toward the second signal.

302 4.5. Capacity controls

303 Step 8 doubles the wait between messages to the same number up to
 304 an hour and caps a number at five a day. Step 9 enforces caps per source,
 305 platform and country, multiplied by a factor between 0.25 and 1.5 that a
 306 baseline job sets from the key’s conversion and its deviation from a four-
 307 week baseline of the pipeline’s own volume counters; the default worker runs
 308 the job every minute, and Section 7.4 shows that the cadence is not a detail.
 309 A tightened cap does not ration a client with verified history. Step 10 reserves
 310 the hourly message count and spend before a send and refuses the send if
 311 either budget is exhausted, which is a hard ceiling; below it, utilisation above
 312 80 % moves the pipeline to an elevated mode and 100 % to an emergency mode
 313 in which only the cleanest traffic uses SMS.

314 The defaults and the reasons we chose them are in Appendix A; the
 315 pseudocode for each step is in the repository [22].

316 5. An estimate of pumping leakage

317 Against the block tests, a pumper’s leakage depends not on how many
 318 numbers it uses but on how many blocks it touches. Let B be the number
 319 of distinct 8-digit blocks its requests actually reach, λ its request rate, N
 320 its total requests, and k the number of sends a block costs before its verdict
 321 when every send goes the attacker’s way and the block starts with no history.
 322 With the parameters above $k = \lceil 6.91/1.50 \rceil = 5$ for both tests. Then

$$L \approx \min(N, kB + \lambda \tau), \quad (2)$$

323 where τ is the time a send stays unresolved: for a carrier that never verifies,
 324 the resolution window plus the delivery delay plus half the period of the
 325 worker that resolves timeouts, about 2.5 minutes with our defaults; for one
 326 that verifies at once, the delivery delay plus its own entry delay. Equation (2)
 327 is an approximation under stated conditions, not a bound: it assumes empty
 328 history, no credit, no solved challenges, no exemptions and no suspension.
 329 Section 7.5 reports its error at nineteen configurations.

330 A carrier that verifies a share σ of its codes with human-like delay changes
 331 the conversion statistic’s expected increment per send to $\sigma(-2.08) + (1 -$

332 $\sigma)(1.50)$, which is zero at $\sigma^* = 0.42$. Below that share the statistic drifts up-
 333 ward and a verdict is a matter of time; above it the statistic drifts downward
 334 and the floor decides whether the block is ever caught. This is a zero-drift
 335 point under a Bernoulli model, not a guarantee in either direction, and the
 336 bounded floor is what makes the region above σ^* depend on the carrier's
 337 strategy; Section 7.5 measures one point, 60 %, under all three floors. Each
 338 verified code is also a registered account on a number the pumper controls,
 339 so evasion by verification costs the pumper at least σ fake accounts per paid
 340 message and leaves the defender holding them.

341 **6. Implementation and evaluation method**

342 *6.1. Implementation*

343 The pipeline, store, feedback loop and vendor adapters are about 3 000
 344 lines of Python with one method per step; state sits behind one interface
 345 with an in-memory backend for tests and a Redis backend for deployment.
 346 Adapters exist for reCAPTCHA (including the interactive challenge), Play
 347 Integrity, App Attest, two IP-intelligence services, a number-lookup service,
 348 Twilio SMS and WhatsApp, push and alerts, each with a stated outage
 349 policy: CAPTCHA and attestation fail closed, IP intelligence and number
 350 lookup fail open. A FastAPI service exposes session, request, verify and
 351 delivery-receipt endpoints; the default worker resolves timeouts and runs the
 352 baseline job from the pipeline's own counters. The verification code travels
 353 in the message body, and delivery and verification callbacks are idempotent.
 354 The suite has 227 tests on both backends, including concurrency tests for the
 355 per-number claim and the budget reservation. No live vendor call has been
 356 made from the repository.

357 *6.2. Simulation*

358 The evaluation is a minute-by-minute simulation of one registration source,
 359 with the pipeline running as deployed and the feedback loop driven by the
 360 simulated provider's receipts and the simulated users' entries. Each run
 361 warms up with ten minutes of legitimate traffic (130 minutes for the pump-
 362 ing study, which includes a variant that needs the history), then runs twenty
 363 minutes of attack with legitimate traffic in the background at 20 requests per
 364 minute. Simulated users verify 80 % of codes after a lognormal entry delay
 365 (median 22 s from delivery; median 2 s for the 20 % whose device fills the code
 366 in); 20 % of requests come from a browser that verified before; 70 % of users

are reachable on WhatsApp, the downgrade channel for someone with no registered device; 5 % leave through an in-country hosting range, 5 % roam from abroad and 2 % sit behind a cloud proxy abroad; 90 % complete an interactive challenge when shown one. Receipts arrive after a lognormal delay (median 3 s) plus the sender’s queue delay. Sessions are obtained through the same CAPTCHA gate as the real endpoint; a challenged party retries with the same session, address and number; events are processed at their own timestamps.

Prices are list prices: 0.1422 USD per SMS to Saudi Arabia, 0.008 USD per number lookup, 0.001 USD per CAPTCHA assessment, 0.003 USD per solved token, 3 USD per gigabyte of residential proxy traffic. The behavioural figures, conversion, entry delay, autofill share, returning share, fingerprint freshness, challenge completion and CAPTCHA score distributions, are assumptions, not measurements; the calibration file marks each one and gives the published figure it was set against where one exists (one large provider reports a 68 % global conversion rate [23]). Every absolute number in Section 7 is conditional on these assumptions.

The random streams are split: the offered workload (arrivals, identities, numbers, entry delays) is drawn at request creation independently of the defence, so runs with the same seed see the same traffic and differences between designs can be paired per seed. Each main-study attacker runs on thirty seeds, with the pool size (500 to 50 000 addresses), the request rate (10 to 60 per minute) and the CAPTCHA score class drawn per seed; sweeps, the ablation and the pumping study use ten seeds, the dilution curve five, the 24-hour study five per cell and the outage study eight. Results are means with 95 % *t*-intervals over seeds. Two modes are run: *caps lifted*, so that the other layers are visible, and *with source caps* at three times the legitimate rate, adaptive for v2 and static for designs without Step 9.

6.3. Designs compared

Seven designs run through the same code with feature flags: **v1**, the first design with the header-trusted platform and static caps; **v1-corrected**, v1 with the platform derived from attestation; **budget-only**, attestation, sessions, static caps and the hard hourly budget, no scoring; **block-limit-only**, attestation, sessions and a flat limit of five sends per destination block per day, no scoring; **conversion-only** and **speed-only**, v2 with one of the two block tests switched off; and **v2**.

403 6.4. *Metrics*

404 For the attacker: messages leaked over the twenty minutes; the fraction
405 of seeds contained, where contained means leakage fell to 5% of the attack
406 rate and stayed there to the end of the run; time to containment averaged
407 over contained seeds only; late-window leakage per minute after containment,
408 or over the last five minutes of an uncontained run, which is a late-window
409 rate and not a steady state. For simulated users: dispatched (a channel was
410 chosen), delivered (the provider’s receipt said so), completed (the code was
411 entered), challenged and refused, each as a share of their requests, reported
412 for all users and separately for first-time clients (no verified history) and
413 returning clients; a failed delivery counts as refused. For the block tests:
414 blocks that reached a verdict and the share of real users a verdict touched.

415 6.5. *Attackers*

416 Table 4 lists the profiles. The main study’s nine cover the incident’s
417 attacker and its natural variants. The adaptive attackers were added one
418 review round at a time as the evaluation found gaps: a challenge solver, low-
419 and-slow profiles under the dilution bound, a pumper that builds verified
420 history before flooding, a carrier that reports every delivery as failed, and
421 an attacker that floods the blocks real users concentrate on in order to earn
422 them a verdict. The pumping study’s carriers differ only in how they handle
423 the codes delivered to them.

Table 4: Attacker profiles. Residential attackers draw addresses from in-country residential ranges; “farmed” CAPTCHA scores follow the human distribution.

Profile	Description
<i>Main study</i>	
naive single client	One address, one session, random numbers, a basic bot’s CAPTCHA scores
datacenter rotation	Hosting ASN abroad, fresh fingerprint per request, headless-browser scores
residential bot	Residential pool, fresh fingerprint, bot or headless scores
residential CAPTCHA farm	Residential pool, fresh fingerprint, farmed scores
residential aged fingerprints	As above with fingerprints seeded two hours earlier
residential reused profile	Residential pool, one 48-hour-old browser profile reused
sequential numbers	Residential pool, numbers walked upward
premium pumping	Residential pool, a premium-rate prefix
forged platform header	The CAPTCHA farm sending the iOS header without attestation, with a valid host and session
<i>Adaptive</i>	
pumper, listed range	Colluding carrier on an elevated-cost range; verifies every code within 1 s, or 60 % after 30 s
verifying flooder	Random numbers across all standard prefixes, 60 % verified after 30 s (earns nothing)
challenge solver	Datacenter rotation that buys a solution to every interactive challenge
low and slow	2 or 10 requests per minute over 20 residential ASNs, aged fingerprints
trust-building pumper	500 identities and numbers verify everything for 10 minutes, then flood without verifying
receipt-faking carrier	Concentrated pumper whose carrier reports every delivery as failed
block poisoner	Floods the blocks real users concentrate on, to earn them a verdict
<i>Pumping study</i>	
concentrated pumper	Three 10 000-number blocks inside a standard prefix; the carrier does not verify, verifies within 1 s, verifies 60 % after 30 s, or buys challenge solutions

7. Results

7.1. RQ1: what is contained, and at what cost

Table 5: Messages leaked in twenty minutes (about 600 requests, varying per seed) under five of the seven designs, caps lifted and with source caps, and the share of first-time simulated users refused with caps on; means over thirty seeds. v1-corrected equals v1 except on the forged-header row (563 leaked, 4 % refused); conversion-only and speed-only equal v2 on every row shown. Intervals are in the repository.

Attacker	caps lifted: leaked			with source caps: leaked / first-time refused				
	v1	block-limit	v2	v1	budget	block-limit	v2	
Naive single client	28	2	2	28 / 1 %	2 / 1 %	2 / 1 %	2	1 %
Datacenter rotation	410	410	0	396 / 2 %	396 / 2 %	396 / 2 %	0	1 %
Premium-prefix pumping	554	5	0	550 / 4 %	548 / 5 %	5 / 1 %	0	1 %
Sequential numbers	596	5	71	563 / 4 %	563 / 4 %	5 / 1 %	65	3 %
Residential, reused profile	591	591	128	565 / 3 %	565 / 3 %	565 / 3 %	95	7 %
Residential, bot CAPTCHA scores	217	217	214	217 / 1 %	217 / 1 %	217 / 1 %	152	15 %
Residential, farmed CAPTCHA	591	591	595	563 / 4 %	563 / 4 %	563 / 4 %	243	51 %
Forged platform header	594	591	595	100 / 1 %	563 / 4 %	563 / 4 %	243	51 %

Table 5 and Figure 1 answer the first question. Two attacks leak nothing under v2 at any seed: datacenter rotation, because the hosting ASN, the fresh fingerprint and the foreign address put every request in the challenge tier and this attacker does not solve challenges; and premium-prefix pumping, because Step 5 refuses the range. The single client is held to two messages by the per-session cap. The sequential walk leaks 65 to 71 messages before its block reaches a verdict at minute 3, and the reused browser profile leaks 95 to 128 before its fingerprint’s own history catches it at minute 4.7. In all of these, 99 % or more of simulated users are delivered with the caps lifted, and 95 to 99 % with caps on; about 2 % are challenged, which the generator attributes to its corporate, roaming and cloud-proxy minority. The aged-fingerprint attacker behaves exactly as the CAPTCHA farm in every table and is omitted from them.

Two simpler designs set what the sequential tests are worth. The budget-only design contains the naive client and nothing else: a hard budget bounds spend, it does not separate. The flat limit of five sends per block per day contains the sequential walk and the premium pumper at five messages each, better than v2’s 65 to 71 on the walk, and nothing else. Against attacks that stay inside a few blocks and never verify a counter is enough; the sequential tests earn their place only against carriers that verify (Section 7.5).

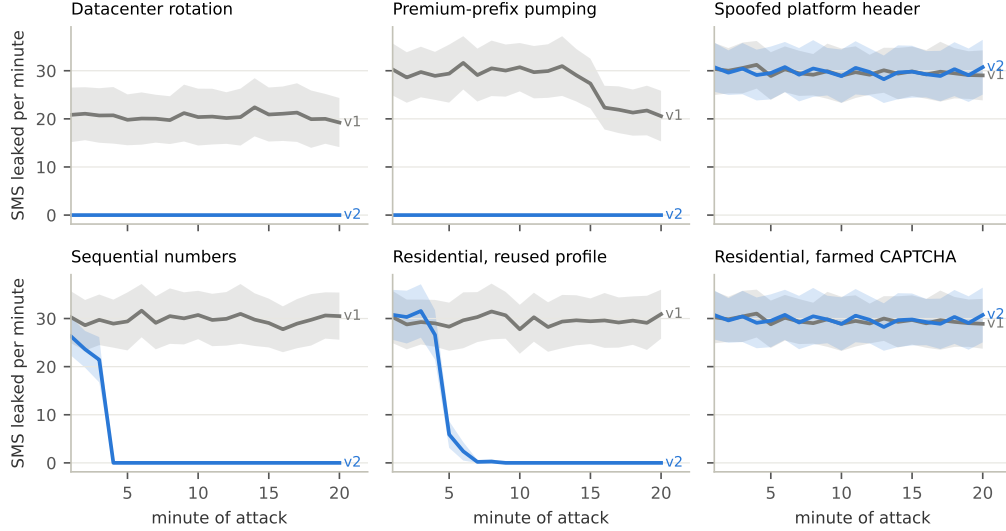


Figure 1: Messages leaked per minute of attack, v1 against v2, mean over thirty seeds with the caps lifted; the shaded band is the 95 % interval of the mean. Zero from the first minute (datacenter, premium) is a different outcome from containment after a few minutes (sequential, reused profile); the farmed-CAPTCHA attacker is unaffected.

446 The forged-header row is the one whose story changed under review.
 447 Under v2 the header buys the attacker nothing: it is ignored, the request is
 448 judged as web traffic, and the row equals the farm's. Under v1 with caps on,
 449 the header is believed, the attacker lands in the app bucket, and the static
 450 app cap holds it to 100 messages without touching web users. Attestation
 451 removes the attacker's choice of bucket; it does not by itself reduce leakage,
 452 and the paired ablation below says the same.

453 The last rows are the other half of the answer. A residential attacker
 454 with farmed scores, fresh or pre-aged fingerprints and valid numbers leaks
 455 everything with the caps lifted under every design, 591 to 595 of about 600,
 456 and the intervals overlap. With caps on, v1's static cap leaks 563 and refuses
 457 4 % of first-time users; v2's adaptive cap leaks 243 and refuses 51 % of first-
 458 time users, while 99.5 % of returning users are delivered and 60 % of all
 459 simulated requests are. All of v2's gain against this attacker is rationing,
 460 and Section 7.4 shows it depends on one setting.

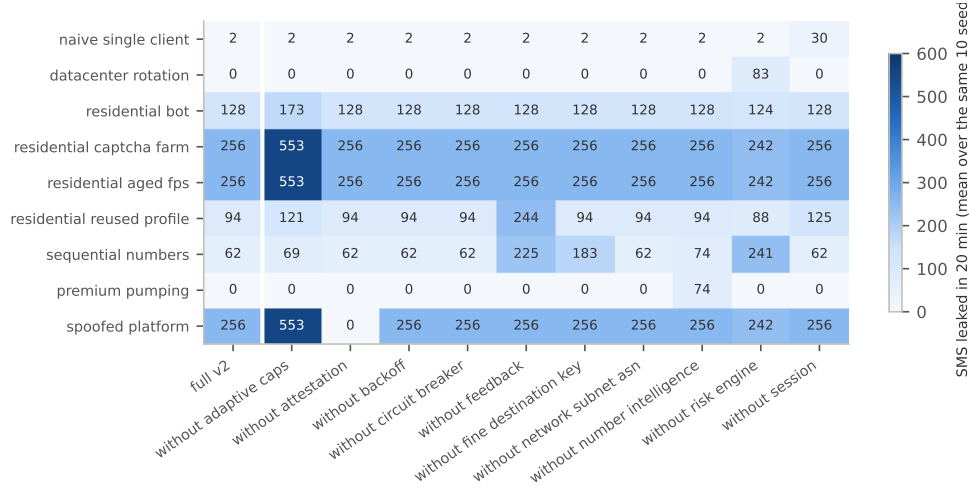


Figure 2: Leave-one-layer-out ablation with source caps on, same ten seeds and same offered workload in every cell; the first column is the complete design. A layer that blocks the same requests as another shows no difference, so a cell is a marginal, not a total, contribution.

7.2. RQ2: which layer does what

Figure 2 removes one layer at a time on the same workload; Table 6 gives the paired differences with the user-side effect the heatmap omits. Each signature attack has one layer that stops it: the session cap for the naive client (+28 messages without it, no change in delivery), the risk engine for datacenter rotation (+83, and -3.9 points of delivery, because the attacker then consumes the adaptive cap), number intelligence for premium pumping (+74). The feedback loop stops the reused profile (+151 without it, and delivery falls by 35 points as the cap tightens on everyone) and, with the block key, the sequential walk (+162 without the loop, +120 without the block key). Against the two farmed-CAPTCHA attackers one layer changes anything: without the adaptive cap they leak +297 and delivery rises by 34 points. Removing attestation under the forged-header attacker *lowers* leakage by 256 and raises delivery by 37 points, because the believed header moves the attacker into the app bucket; this is the bucket effect of Section 7.1, not a reason to remove attestation, since an attacker that can choose its bucket chooses the one that costs it least.

Table 6: Paired per-seed differences against the complete design, same seeds and workload: messages leaked and simulated-user delivery, with 95 % intervals. Rows where neither interval excludes zero are omitted.

Attacker	Layer removed	Leaked, difference	Delivery, difference (points)
naive single client	session	+28 [+12, +44]	0.0
datacenter rotation	risk engine	+83 [+68, +99]	-3.9 [-6.4, -1.4]
CAPTCHA farm	adaptive cap	+297 [+142, +452]	+34.0 [+19.3, +48.7]
reused profile	feedback loop	+151 [+133, +169]	-35.4 [-45.6, -25.2]
reused profile	session	+32 [+18, +45]	-7.0 [-10.1, -3.8]
sequential numbers	risk engine	+179 [+161, +196]	-36.7 [-48.9, -24.4]
sequential numbers	feedback loop	+162 [+148, +177]	-28.9 [-43.7, -14.2]
sequential numbers	block key	+120 [+98, +143]	-7.3 [-14.4, -0.3]
premium pumping	number intelligence	+74 [+52, +95]	-2.4 [-3.8, -1.0]
forged header	attestation	-256 [-272, -241]	+36.7 [+21.0, +52.4]

478 7.3. RQ3: where reputation stops separating

479 The conversion penalty attaches to the keys a request touches. A residen-
480 tial attacker with farmed scores touches an address it uses once, a fingerprint
481 it uses once, and a country, a prefix and a residential ASN it shares with
482 every real user. Its failures land on the shared keys, and the penalty they
483 produce lands on everyone: that is rationing, not separation. Three experi-
484 ments show how far this goes within this design. We do not claim more than
485 that: a finite sweep cannot show that no setting separates, and we have not
486 proved an indistinguishability result. What follows is the evidence that the
487 signals this design observes do not do it.

488 Figure 3 raises the attack from half the legitimate rate to ten times it for
489 an hour. Up to twice the legitimate rate the attack leaks 99 to 100 % because
490 the shared keys’ ratios stay above 0.3 while real users keep verifying. At five
491 times, 95 % leaks while 7 % of real users are challenged and 2 % refused; at
492 ten times, 70 % leaks while 17 % are challenged and 3 % refused. The penalty
493 starts near twice the legitimate volume and fires on real users as it starts.

494 Table 7 varies the weights, the tier boundaries and the resolution window.
495 The conversion weight and the resolution window change nothing, because
496 the conversion signal does not fire on these keys at any value tried. Scaling
497 the tier boundaries to 0.6 cuts leakage by about 40 % and challenges 18 %
498 of real users. Raising the fresh-fingerprint weight to 40 stops the attacker
499 outright, challenges 46 % of real users and refuses 8 %, because most real
500 sign-ups are also new browsers. (An earlier version of this table swept the
501 code’s validity instead of the resolution window, which left the window un-
502 changed throughout; the sweep now varies the window itself, with the same
503 null result.)

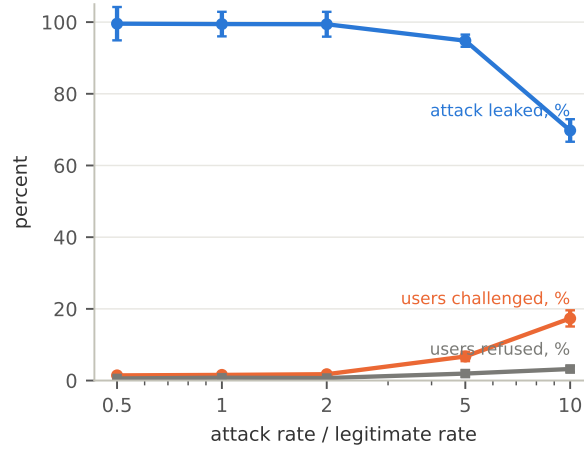


Figure 3: The farmed-CAPTCHA attacker at multiples of the legitimate rate for an hour, caps lifted, five seeds, with 95 % intervals: share of the attack leaked, share of real users challenged and share refused.

Table 7: Weight, boundary and resolution-window sweep against the farmed-CAPTCHA attacker, caps lifted, ten seeds. Defaults: conversion weight 25, fresh-fingerprint weight 20, boundary scale 1.0, resolution window 120 s. The attack size varies per seed; “leaked” is the mean total.

Parameter	Value	Leaked	Challenged	Refused
default	–	576	2.0 %	1.0 %
conversion weight	40	576	2.0 %	1.0 %
resolution window	60, 300, 600 s	576	2.0 %	1.0 %
boundary scale	0.6	358	17.8 %	3.6 %
fresh-fingerprint weight	40	0	46.3 %	8.1 %
weight				

504 What the three experiments say together is that in this design outcome-
505 based reputation separates an attacker only on keys the attacker dominates,
506 and the attacker chooses how pure the client-side keys are. Douceur’s argu-
507 ment explains why: nothing scarce backs a web identity [13]. The keys where
508 separation remains possible are the ones the attacker reuses by mistake, like
509 the reused profile, and the one it cannot rotate, the destination.

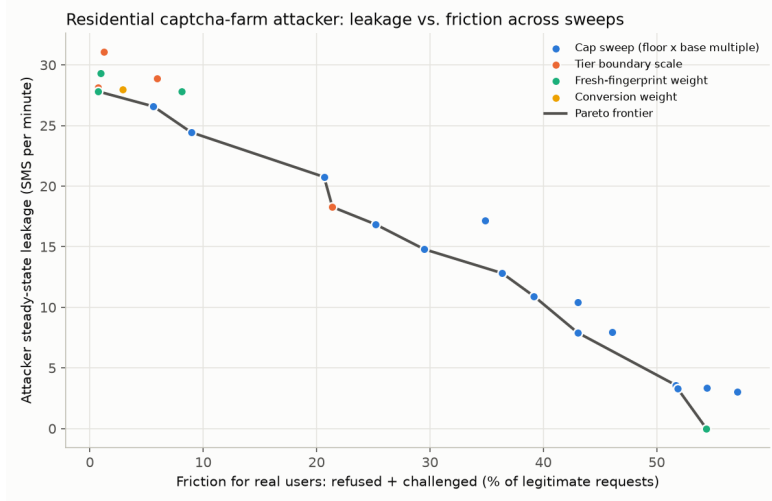


Figure 4: Late-window leakage of the farmed-CAPTCHA attacker against a friction index for simulated users (requests refused plus requests challenged; a user can count under both), over the cap and weight sweeps. The line joins the nondominated sampled settings; intermediate policies were not run.

510 7.4. RQ4: what rationing costs, and what it depends on

511 When separation fails, the volume cap is the lever this design has, and
 512 Figure 4 measures it. Holding the attacker to about 11 messages a minute
 513 refuses 38 % of simulated requests (the default: adaptive floor 0.25, base cap
 514 three times the legitimate rate); holding it to about 3 a minute refuses 56 %;
 515 letting it through refuses 1 %. The frontier is the nondominated set of the
 516 configurations we ran, not an optimum.

517 The gain depends on one thing the first version of this paper did not
 518 report. The baseline job that tightens the cap runs every minute in the
 519 default worker. Run every ten minutes, the same design leaks 565 of the
 520 farm’s messages at 2 % first-time refusal; run hourly, 575 at 1.3 %, which
 521 is the static cap. A twenty-minute attack seen by an hourly controller is
 522 not seen at all. Whether a minute-level controller on a four-week baseline is
 523 operable in a given deployment is a question for that deployment; we state
 524 the dependence rather than resolve it.

525 7.5. RQ3 continued: pumping on the key the attacker cannot rotate

Table 8: Concentrated pumpers on three 10 000-number blocks, caps lifted, ten seeds, 130-minute warm-up: messages leaked of about 600, fraction of seeds contained with the mean containment time over those seeds, verified fake accounts the carrier created, and blocks under verdict. Default: block key, two-minute window, credit floored at one threshold, graded verdicts.

Carrier	Design	Leaked	Contained (min)	Fake accounts	Blocks
does not verify	v1	574	0 of 10	0	0
	v2 default	99	10 of 10 (4.4)	0	2.9
	v2, 24-h denylist	99	10 of 10 (4.4)	0	2.9
verifies within 1 s	v1	574	0 of 10	574	0
	v2 default	36	10 of 10 (2.6)	36	3.0
verifies 60 % after 30 s	v1	574	0 of 10	348	0
	v2, credit floored at one threshold (default)	440	2 of 10 (18.8)	264	1.2
	v2, no credit (Page's CUSUM)	158	9 of 10 (10.3)	93	3.0
	v2, unbounded credit (plain SPRT)	575	0 of 10	344	0
does not verify; buys solutions farmed-CAPTCHA flooder (control)	v2 default	115	10 of 10 (5.2)	0	3.0
	v2, 24-h denylist	102	10 of 10 (4.6)	0	3.0
	v2 default	568	0 of 10	0	0

526 Table 8 is the pumping result. A pumper whose carrier does not verify is
527 contained in every seed, on average by minute 4.4, with 99 messages leaked;
528 one whose carrier enters every code within a second is caught by the speed
529 test by minute 2.6 with 36 leaked. A pumper that buys a solution to every
530 challenge earns a second verdict on its blocks and is moved off SMS, at 115
531 leaked against 102 under the hard denylist; that is what the denylist buys,
532 thirteen messages, against a lockout of 10 000 numbers for a day. The flooder
533 control is untouched, as it should be: it touches blocks at random.

534 The carrier that verifies 60 % of its codes after a human-like delay is the
535 case that decides the detector's memory. With unbounded credit it is never
536 caught and leaks 575. With no credit it is caught in nine seeds of ten, by
537 minute 10, at 158 leaked and 93 fake accounts. With one threshold of credit,
538 the default, it is caught in two seeds of ten and leaks 440, leaving 264 fake
539 accounts. We chose the default anyway, because the no-credit setting pro-
540 duces about a hundred false verdicts a day on a legitimate block converting

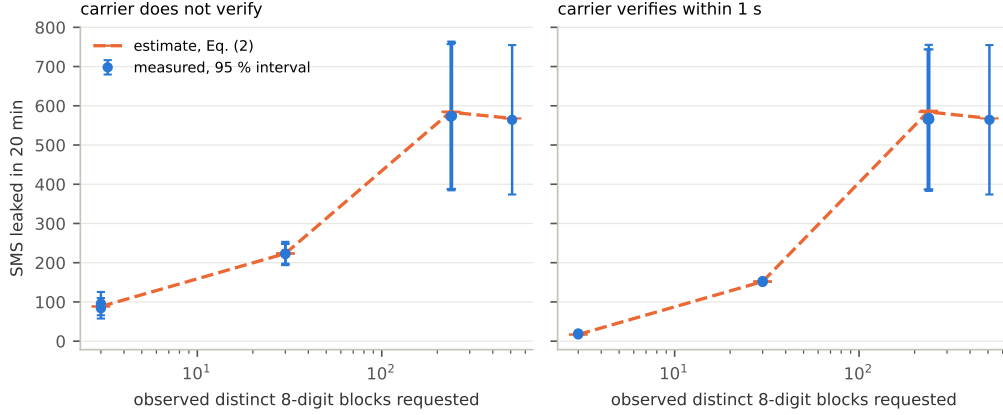


Figure 5: Messages leaked against the observed number of distinct destination blocks a pumper’s requests reached, for ranges that fit inside, align with, or span the 8-digit key; bands are 95 % intervals over ten seeds. Leakage follows the blocks touched, not the numbers used.

541 at 65 % (Section 7.6); the setting is a trade between this carrier and those
542 users, and we report it as one. Two adaptive carriers defeat the tests out-
543 right (Section 7.7): one that reports every delivery as failed feeds the tests
544 nothing, and one that verifies for ten minutes before flooding arrives with
545 exempt identities and trusted numbers.

546 Figure 5 and Table 9 vary how widely the pumper spreads, with ranges
547 drawn without replacement and the number of blocks its requests actually
548 reached recorded per run. Ranges of 1000 and 10 000 numbers behave iden-
549 tically because each sits in one block; ranges of 100 000 numbers touch ten
550 blocks each; and once the attack reaches more than about 200 distinct blocks
551 in 600 requests the tests do not fire in time and the pumper is the flooder
552 again. Equation (2), evaluated on the observed block count, is within 7 mes-
553 sages of the measurement at every configuration up to 30 blocks and within
554 19 at every configuration, with the largest errors where the estimate is sat-
555 urated at N and the measurement is not quite. The in-flight term is what
556 separates the two carriers at small B : the non-verifier’s sends stay unresolved
557 for the two-minute window; the verifier’s resolve within seconds.

Table 9: The leakage estimate against the spread sweep, ten seeds, one configuration per row: nominal pool, observed distinct blocks requested, measured leakage, the estimate on the observed count, and the error.

Carrier	Ranges $\times$ size	Nominal blocks	Observed blocks	Measured	Estimate	Error
does not verify	$3 \times 1\,000$	3	3	84	88	+5
	$3 \times 10\,000$	3	3	96	88	-7
	$3 \times 100\,000$	30	30	223	223	+1
	$30 \times 1\,000$	30	30	224	223	0
	$30 \times 10\,000$	30	30	222	223	+1
	$30 \times 100\,000$	300	240	574	583	+9
	$300 \times 1\,000$	300	236	572	585	+12
	$300 \times 10\,000$	300	238	574	585	+11
	$300 \times 100\,000$	3\,000	509	564	568	+4
verifies within 1 s	$3 \times 1\,000$	3	3	20	17	-3
	$3 \times 10\,000$	3	3	17	17	0
	$3 \times 100\,000$	30	30	151	152	+1
	$30 \times 1\,000$	30	30	153	152	-1
	$30 \times 10\,000$	30	30	153	152	-1
	$30 \times 100\,000$	300	240	565	584	+19
	$300 \times 1\,000$	300	236	565	584	+19
	$300 \times 10\,000$	300	239	569	588	+18
	$300 \times 100\,000$	3\,000	509	564	568	+4

558 7.6. RQ5: what the block tests cost real users

559 The block tests act on a key real people share by the thousand, and they
560 were calibrated to the simulated population. Four experiments ask what they
561 do when the population differs, over a day, during an outage, and when an
562 attacker aims them.

563 The first is a Monte Carlo of the deployed tests, bounded floor included,
564 on one block that sees only legitimate traffic (Table 10). It gives the proba-
565 bility that the block reaches a verdict in a day, which is a different quantity
566 from the 0.2^5 probability of one particular run of five failures: the test runs
567 continuously, restarts after a verdict and sees both statistics. With the tests
568 assuming 80 % conversion and 20 % fast verifications, a population convert-
569 ing at 80 % produces a verdict on about one block in a thousand per day;
570 a 30 % autofill share against the 20 % assumption raises that to under one
571 percent. A population converting at 65 % produces a verdict on about 2 %
572 of blocks with thirty seeds a day and 2.5 to 3.5 % at a hundred; at 50 %, on
573 9 to 41 %. The conversion assumption is the sensitive one, and it must come
574 from measured traffic.

Table 10: Probability that a block seeing only legitimate traffic reaches a verdict in one day, by true conversion and true share of verifications within 5 s of delivery, at 30 sends per block per day; tests as deployed. Monte Carlo, 20 000 trials per cell; a cell of 0.00 % means fewer than one trial in 20 000.

Conversion	fast 0 %	5 %	10 %	20 %	30 %
50 %	19.7 %	19.7 %	19.8 %	19.5 %	20.3 %
65 %	1.9 %	2.0 %	1.9 %	1.9 %	2.6 %
80 %	0.07 %	0.10 %	0.07 %	0.09 %	0.69 %
90 %	0.00 %	0.01 %	0.01 %	0.06 %	0.79 %

575 The second runs the full simulation for 24 hours with no attacker, five
576 seeds per cell, with legitimate numbers drawn from a fixed set of blocks so
577 that each sees 144, 29 or about 4 sends a day, under two protocols: the
578 detector’s speed calibration held at the deployed 20 %, and recalibrated to
579 the population’s actual autofill share. At 80 % conversion the day produces
580 0 to 2 verdicts and touches at most 2 of about 28 700 simulated users in most
581 cells; the worst cells, 30 % autofill against the fixed 20 % calibration, produce
582 5 to 7 verdicts and touch 1 to 5 users, and recalibrating brings them back
583 to 0 to 1. At 65 % conversion the day produces 6 to 21 verdicts and touches
584 20 to 43 users at 144 and 29 sends a day, under either protocol, because the
585 conversion assumption, not the speed one, is what is wrong; those users meet
586 a challenge, and delivery stays at 99.3 % in every cell. With the no-credit
587 floor the same 65 % population produced 103 to 139 verdicts and touched
588 700 to 870 users a day, which is why the bounded floor is the default despite
589 what it costs in Table 8.

590 The third stops one prefix, a seventh of the traffic, for thirty minutes in-
591 side an hour, eight seeds, in two forms: the provider reports every send failed,
592 or reports delivery while nobody receives anything. With receipts gating the
593 tests and the carrier-wide suspension, the outage produces 0.5 to 0.8 verdicts
594 and touches 1 to 2 users; without receipts, 1.5 verdicts and 3 users. The
595 receipts help; they do not remove the effect, and the suspension’s conversion
596 signal needs ten returning users’ failures before it fires. Delivery during the
597 failed-receipt outage is 92 %, which is the outage itself now measured as a
598 delivery, not a dispatch.

599 The fourth is an attacker. The block poisoner floods the blocks real users
600 concentrate on so that those blocks reach a verdict: with caps lifted it earns
601 verdicts on 12 blocks per run and makes 25 % of simulated users solve a
602 challenge; with caps on, 8 blocks and 11 %, with delivery at 96 % and 69 %
603 (the second mostly through the cap). The graded verdict keeps the harm at

604 a challenge; the hard denylist would have refused those users for a day. This
 605 is the central risk of destination-keyed reputation, and we have no mitigation
 606 beyond the grading to report.

607 *7.7. Adaptive attackers, economics and overhead*

608 A colluding carrier on a range the prefix table lists as elevated-cost is
 609 held to 7 messages when it verifies instantly (the speed test) and 20 when it
 610 verifies with human-like delay (the per-prefix cap); on an unlisted standard
 611 range the human-like carrier is the pumper of Table 8. A datacenter attacker
 612 that buys a solution to every challenge leaks 53 to 56 messages per run and
 613 loses about 256 session attempts at the CAPTCHA gate doing it. Low-and-
 614 slow attacks under the dilution bound leak at their own rate: 41 messages in
 615 twenty minutes at two per minute over twenty ASNs, 198 at ten. The trust-
 616 building pumper, whose 500 identities and numbers verify everything for ten
 617 minutes and then flood, leaks 511 with caps lifted and 349 with caps on,
 618 keeping 262 and 194 verified accounts; the exemptions meant for returning
 619 users work in its favour, and nothing in the design distinguishes earned trust
 620 from bought trust. The receipt-faking carrier leaks 567 and 255 with no
 621 verdict ever. Both are bounded only by the caps and the budget.

622 With revenue credited only to pumpers at a 20 to 50 % share of the
 623 0.1422 USD retail price, premium-prefix pumping earns 14 to 38 USD per
 624 twenty minutes under v1 and nothing under v2; under v2 the concentrated
 625 pumper earns 0.85 USD at the 50 % share when it verifies instantly, 5.27
 626 USD when it never verifies, and 29.50 USD plus 264 accounts when it verifies
 627 like a person. This is the stated accounting, which charges the attacker for
 628 requests sent after containment and takes the retail price as the termination
 629 fee; real termination revenue is not public.

630 Overhead was measured against a real Redis on a four-vCPU container
 631 with four worker processes and faked vendors: a full send costs 11.0 ms at
 632 the median and 18.9 ms at the 99th percentile in process, with 48 Redis
 633 round trips, and the service serves 248 requests per second at concurrency
 634 32 without the minimum response time. With it, 6 000 requests across five
 635 outcome classes including the challenge showed no pair that a Kolmogorov-
 636 Smirnov test distinguished (smallest $p = 0.32$) and every pair's mean within
 637 2 ms by a two one-sided test (largest difference 0.22 ms); without it, nine
 638 of ten pairs are distinguished. At this sample size and with faked vendor
 639 latency, one response time tells an attacker nothing about the outcome; a
 640 larger sample, real vendor latency or another statistic might.

641 8. Discussion: lessons and limitations

642 8.1. Lessons

- 643 1. *In this design, reputation separates attackers only on keys they domi-*
644 *nate.* The dilution curve, the sweep and the cadence study all say so,
645 and Douceur’s argument says why. We state it as an empirical result
646 for the signals this design observes, not as a theorem.
- 647 2. *A counter is enough for a pumper that never verifies.* The flat per-
648 block limit matched or beat the sequential tests on the walk and the
649 premium range. The tests pay for themselves only against carriers
650 that enter codes, and their memory setting decides which of those they
651 catch.
- 652 3. *The detector’s memory is a trade between a verifying carrier and legit-*
653 *imate blocks.* No credit catches the 60 % verifier and flags a 65 % block
654 a hundred times a day; one threshold of credit does the reverse. There
655 is no setting that does both, and the choice belongs to the deployment.
- 656 4. *A wrong verdict on a shared key must be graded, and an attacker can*
657 *cause one.* The poisoner made a quarter of simulated users solve a
658 challenge. Grading kept it at that.
- 659 5. *Failure is a delivery receipt and a timeout, and the receipt comes from*
660 *the adversary’s side of the network.* Clocking from the receipt removed
661 the outage false positives the send clock produced; a carrier that fakes
662 receipts removes the detector.
- 663 6. *Rationing needs a fast controller, and the block tests need a measured*
664 *calibration.* The adaptive cap’s effect came entirely from a minute-level
665 job; the tests assume a conversion rate and an autofill share, and set
666 wrong they flag real blocks at the rates of Table 10.

667 8.2. Limitations

668 Everything is simulated, and the behavioural calibration is assumed, not
669 validated: conversion, entry delays, autofill and returning shares, fingerprint
670 freshness, CAPTCHA score distributions and challenge completion are all
671 choices, and the confidence intervals cover seeds, not those choices. No logs
672 from the incident or the v1 period exist in our hands; the repository has a
673 replay tool and a log schema for when they do, and that replay is the exper-
674 iment this paper most needs. The number-lookup fake says every number is
675 live. The prefix table is a sample. Legitimate behaviour is stylised: one re-
676 quest per user, no retries after a failed delivery, every user on the attacker’s

677 ISP, no synchronised bursts, no multi-day attacks, no device farms. The
 678 adaptive controller’s four-week baseline is replaced by the known legitimate
 679 rate. Non-SMS channels are assumed available for 70 % of users; for the
 680 rest a downgrade is a refusal. Attackers do not adapt within a run except
 681 through the adaptive profiles, and two of those, the receipt-faking carrier and
 682 the trust builder, defeat the block tests outright. The paired ablation mea-
 683 sures marginal, not total, contributions. We did not evaluate client puzzles, a
 684 sliding-window or change-detection alternative to the cumulative statistic, a
 685 supervised model, or a time-uniform test of the kind Howard et al. give; each
 686 is a sensible comparison we lack. The timing measurement excludes vendor
 687 latency and Step 0’s 403 responses. The leakage estimate is conditional on
 688 empty history, no credit and no exemptions, and its error is reported, not
 689 bounded.

690 *8.3. Ethics and privacy*

691 The design hashes browser attributes and keeps the hash’s first-seen time
 692 for 90 days; keeps addresses and numbers in reputation counters for a day
 693 and in audit records for thirty; keeps the set of numbers that verified once
 694 until it is cleared; and challenges people on the strength of what strangers
 695 on their network or in their number block did. Each is a choice with a
 696 cost, and a deployment under the GDPR or Saudi Arabia’s PDPL needs a
 697 lawful basis, a retention statement and a route for a refused person to reach
 698 someone. No human data was used; the simulated users are synthetic. The
 699 incident is described from the operator’s recollection without identifying the
 700 operator, its product or its country beyond the number ranges in the sample
 701 configuration, and the repository records that written permission to describe
 702 it is being sought.

703 **9. Conclusion**

704 An OTP endpoint is an SMS budget behind a form, and the first thing
 705 that stops a flood is not the last. In the simulation reported here, the design
 706 contains the attacks that leave a signature in the request at no measured
 707 cost to simulated users, and it contains a pumper concentrated on a few des-
 708 tination blocks at a cost that a closed-form estimate reproduces to within
 709 19 messages across nineteen configurations. It does not separate a residen-
 710 tial attacker that looks like a person: no signal in the sweep did, the only
 711 containment is a volume cap that refuses half of first-time users, and that

cap works only when recomputed every minute. The destination tests are defeated by a carrier that verifies enough of its own codes, builds trust first or fakes its receipts, and they can be turned on real users by an attacker who floods their blocks. Their memory setting is a measured trade between a verifying carrier and false verdicts on legitimate blocks. Those are the results; the artifact lets anyone rerun them, and replaying real traffic through it is the experiment that would decide how much of this survives contact with a real population.

Artifact availability

The implementation, the simulation, every recorded result and the scripts that regenerate them are at <https://github.com/nahmedpsu/OTP-DDOS>, release 2.5.0 (an archived DOI will be added on acceptance). `make results` regenerates the evaluation tables from fixed seeds, the figure script in `paper/` draws every figure from the recorded runs, and `make load-test` reruns the performance measurement, whose absolute numbers depend on the machine. The full tables with intervals are in the `results` directory (`evaluation.md`, `performance.md`).

CRedit authorship contribution statement

Naveed Ahmed: Conceptualization, Methodology, Software, Validation, Investigation, Writing – original draft, Writing – review & editing. TODO: add co-authors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author used Claude (Anthropic) to assist with writing and reviewing code, running the evaluation, and drafting and editing the manuscript text. The author reviewed and verified every number, claim and reference, and takes full responsibility for the content of the publication.

744 **Data availability**

745 No real traffic data was used. All synthetic data, seeds and results are in
746 the repository above.

747 **Appendix A. Default values and the reasons we chose them**

748 The reasons are design judgements, not findings, except where a section
749 is cited. The repository lists every other default.

Setting	Default	Reason
Tier boundaries	20 / 40 / 60 / 80	A clean web user scores under 15; the largest shared-key signal alone lands in <i>de-lay</i>
Conversion penalty	below 0.3, at least 20 resolved sends	Well under any legitimate conversion we expect; twenty sends keeps one user's typos from triggering it
Resolution window	120s after delivery	99 % of simulated users verify within about 100s; a late verification is reclassified
Block tests	threshold $\ln 1000$; $p_\ell = 0.8$, $p_a = 0.1$; $q_\ell = 0.2$, $q_a = 0.9$; 5s from receipt; floor at one threshold	False-verdict rates in Table 10; the floor trades Table 8 against Section 7.6
Block verdict	challenge for 1 h, then non-SMS; clients with verified history exempt	Bounds a wrong verdict to first-time users for an hour
Outage suspension	15 min; 10 blocks; half of 20 sends undelivered, or returning users under 30 % on 10 sends in 30 min	Only returning users count, so a decoy flood cannot buy it
Hourly budget; minimum response time	twice the p99 legitimate hour; 400 ms	Room for the elevated mode before the hard ceiling; above the wall-time p99 with one worker per vCPU
Per-number cap; session cap; controller cadence	5 per day; 3 per 10 min; every minute	A real user retries two or three times; Section 7.4 for the cadence

References

- [1] MITRE Corporation, Resource hijacking: SMS pumping, technique T1496.003, MITRE ATT&CK, <https://attack.mitre.org/techniques/T1496/003/>, accessed 1 October 2026.
- [2] Commsrisk, Elon Musk says Twitter lost \$60mn a year because 390 telcos used bot accounts to pump A2P SMS, <https://commsrisk.com/elon-musk-says-twitter-lost-60mn-a-year-because-390-telcos-used-bot-accounts-to-pump-a2p-sms/>, trade press report of a public statement; the figure is the company’s own claim (2023).
- [3] J. H. Huh, H. Shin, S. Ahn, H. Yi, J. Cho, T. Kim, M. Lim, N. Choi, Preventing artificially inflated SMS attacks through large-scale traffic inspection, in: 34th USENIX Security Symposium, USENIX Association, Seattle, WA, 2025.
- [4] M. Sahin, A. Francillon, P. Gupta, M. Ahamad, SoK: Fraud in telephony networks, in: 2017 IEEE European Symposium on Security and Privacy (EuroS&P), IEEE, 2017. doi:10.1109/EuroSP.2017.40.
- [5] M. Sahin, A. Francillon, Understanding and detecting international revenue share fraud, in: Network and Distributed System Security Symposium (NDSS), Internet Society, 2021. doi:10.14722/ndss.2021.24051.
- [6] Twilio Inc., Verify fraud guard, <https://www.twilio.com/docs/verify/preventing-toll-fraud/sms-fraud-guard>, product documentation. Accessed 1 October 2026.
- [7] Twilio Inc., SMS pumping risk score (Lookup v2), <https://www.twilio.com/docs/lookup/v2-api/sms-pumping-risk>, product documentation. Accessed 2 October 2026.
- [8] M. Motoyama, K. Levchenko, C. Kanich, D. McCoy, G. M. Voelker, S. Savage, Re: CAPTCHAs – understanding CAPTCHA-solving services in an economic context, in: 19th USENIX Security Symposium, USENIX Association, 2010.
- [9] A. Dionysiou, E. Athanasopoulos, SoK: Machine vs. machine – A systematic classification of automated machine learning-based CAPTCHA

- 781 solvers, *Computers & Security* 97 (2020) 101947. doi:10.1016/j.cose
782 .2020.101947.
- 783 [10] D. Dean, A. Stubblefield, Using client puzzles to protect TLS, in: 10th
784 USENIX Security Symposium, USENIX Association, 2001.
- 785 [11] X. Mi, X. Feng, X. Liao, B. Liu, X. Wang, F. Qian, Z. Li, S. Alrizah,
786 L. Chen, Y. Qian, Resident evil: Understanding residential IP proxy
787 as a dark service, in: 2019 IEEE Symposium on Security and Privacy
788 (S&P), IEEE, 2019. doi:10.1109/SP.2019.00011.
- 789 [12] P. Richter, F. Wolsing, A. Chandrasekaran, R. Padmanabhan, M. All-
790 man, R. Bush, A. Feldmann, A multi-perspective analysis of carrier-
791 grade NAT deployment, in: Proceedings of the 2016 ACM Internet
792 Measurement Conference (IMC), ACM, 2016. doi:10.1145/298744
793 3.2987474.
- 794 [13] J. R. Douceur, The Sybil attack, in: Peer-to-Peer Systems (IPTPS
795 2002), Vol. 2429 of LNCS, Springer, 2002, pp. 251–260. doi:10.100
796 7/3-540-45748-8_24.
- 797 [14] A. Wald, Sequential tests of statistical hypotheses, *The Annals of Math-*
798 *ematical Statistics* 16 (2) (1945) 117–186. doi:10.1214/aoms/1177731
799 118.
- 800 [15] E. S. Page, Continuous inspection schemes, *Biometrika* 41 (1/2) (1954)
801 100–115. doi:10.2307/2333009.
- 802 [16] S. R. Howard, A. Ramdas, J. McAuliffe, J. Sekhon, Time-uniform, non-
803 parametric, nonasymptotic confidence sequences, *The Annals of Statis-*
804 *tics* 49 (2) (2021) 1055–1080. doi:10.1214/20-AOS1991.
- 805 [17] S. Wiefeling, L. Lo Iacono, M. Dürmuth, Is this really you? An empirical
806 study on risk-based authentication applied in the wild, in: ICT Systems
807 Security and Privacy Protection (IFIP SEC 2019), Vol. 562 of IFIP Ad-
808 vances in Information and Communication Technology, Springer, 2019,
809 pp. 134–148. doi:10.1007/978-3-030-22312-0_10.
- 810 [18] C. Mulliner, R. Borgaonkar, P. Stewin, J.-P. Seifert, SMS-based one-
811 time passwords: Attacks and defense, in: Detection of Intrusions and
812 Malware, and Vulnerability Assessment (DIMVA 2013), Vol. 7967 of

- 813 LNCS, Springer, 2013, pp. 150–159. doi:10.1007/978-3-642-39235-1_9.
814
- 815 [19] H. Siadati, T. Nguyen, P. Gupta, M. Jakobsson, N. Memon, Mind your
816 SMSes: Mitigating social engineering in second factor authentication,
817 Computers & Security 65 (2017) 14–28. doi:10.1016/j.cose.2016.
818 09.009.
- 819 [20] Google LLC, SMS retriever API overview, <https://developers.google.com/identity/sms-retriever/overview>, developer documenta-
820 tion. Accessed 1 October 2026.
821
- 822 [21] Web Incubator Community Group, WebOTP API, <https://github.com/WICG/WebOTP>, specification repository. Accessed 1 October 2026.
823
- 824 [22] N. Ahmed, OTP Guard: a layered SMS validation, risk-scoring and
825 rate-limiting pipeline against OTP flood abuse (version 2.5.0), <https://github.com/nahmedpsu/OTP-DDoS>, source code, evaluation and
826 recorded results. Archived DOI to be added (2026).
827
- 828 [23] Twilio Inc., Verify API, <https://www.twilio.com/en-us/user-authentication-identity/verify>, product page stating a 68%+ global
829 conversion rate. Accessed 1 October 2026.
830